PROJECT MANIFEST

Fast Single-Binary Python CLI Runtime

A compressed, self-contained Python executable whose startup work scales with the executed import path - not with the total dependency set.

> **Binary size may scale with dependencies. Startup work must not. Disk size may scale with compressed dependencies. Decompression work must only scale with resources actually used.**

Design focus: fast CLI startup, single-file distribution, lazy Python imports, lazy native extraction, content-addressed cache, zstd-compressed resource payloads.

1. Mission

Build a packaging and runtime system for Python CLI applications that produces a **single distributable executable** with startup behavior as close as practical to a typical Go or Rust CLI.

The executable contains the Python runtime, application code, third-party packages, resources, native extensions, and required non-system native libraries. Normal operating-system runtime components such as the dynamic loader, glibc, and deliberately supported platform libraries may remain external.

User experience: copy one executable, mark it executable where necessary, and run it. No Python installation, virtualenv, package installation, bootstrap step, or user-visible dependency management.

2. Architectural invariants

Single distribution artifact

One ELF, Mach-O, or PE file contains everything the application needs apart from explicitly allowed system dependencies.

Lazy by construction

Packaging a module never implies importing, decompressing, or extracting it during startup.

Native compatibility first

Use the operating system's normal dynamic loader. Native extensions are materialized only when imported.

Compression is granular

Zstd applies to resource payloads in independently addressable frames. The whole executable is never compressed as one stream.

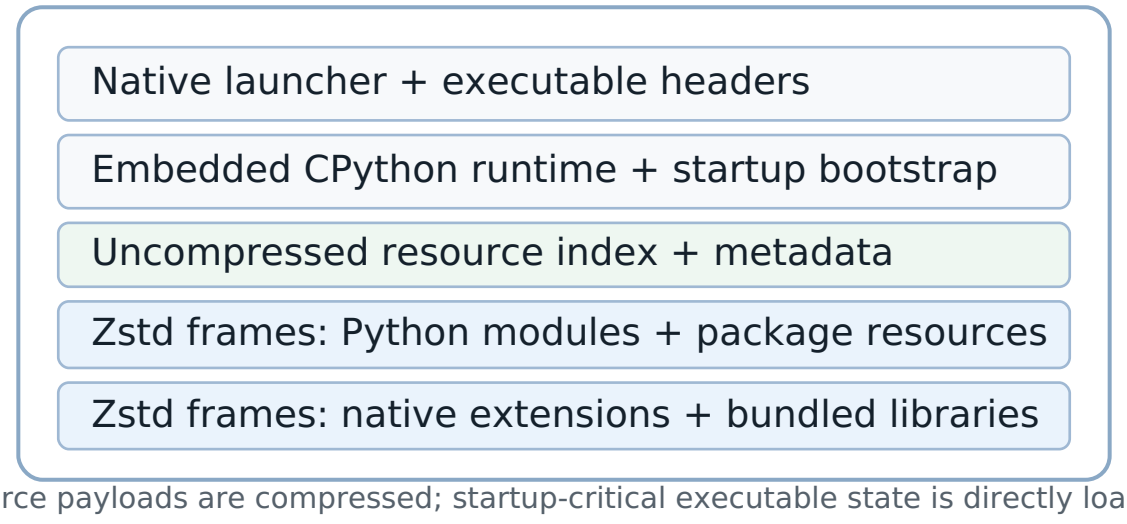


Figure 1. Proposed executable structure. Compression is confined to resource payloads and remains independently addressable.

3. Startup performance contract

The runtime optimizes the cold command path rather than simply reducing distribution size. Normal startup must not unpack the application, scan installed packages, construct an environment, invoke a package manager, extract the standard library, or initialize unused third-party libraries.

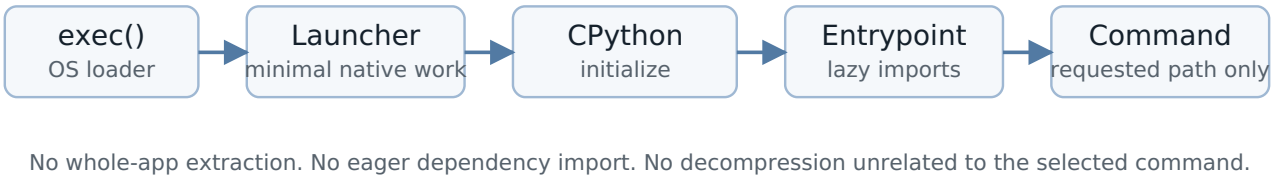


Figure 2. The common startup path remains short and independent of the total packaged dependency set.

Startup target

For a lightweight CLI command, startup cost should be dominated by OS executable loading, CPython initialization, entrypoint imports, and CLI parsing. A large optional package set must have near-zero CPU impact until imported.

4. Resource compression model

Zstandard (zstd) is the default compression format for embedded resource payloads. The runtime should optimize for decompression speed and low latency rather than the highest possible compression ratio.

- The executable itself is not wrapped in a whole-file self-extractor.
- Python modules, package data, native extensions, and bundled libraries are represented as independent or logically grouped zstd frames.
- The resource index remains uncompressed, or uses only a startup-negligible representation.
- Very small startup-critical modules may remain uncompressed when measurement shows that is faster.
- Zstd dictionaries may be supported when they measurably improve size without harming startup latency.

Compression rule: if a resource is never requested, it is never decompressed.

Resource index

Field	Purpose
Name / resource ID	Import name, package resource path, or native artifact identifier.
Type	Python module, package, native extension, native library, ordinary resource, filesystem-required resource.
Offset + compressed size	Direct random access into the executable.
Uncompressed size	Allocation and decompression validation.
Compression	Typically zstd; may be none for startup-critical resources.
Digest	Integrity check and cache identity.
Flags	Startup-critical, filesystem-required, executable, architecture/ABI metadata, etc.

5. Pure-Python import path

Pure-Python modules execute directly from resources embedded in the executable. They are never extracted simply to satisfy Python imports.

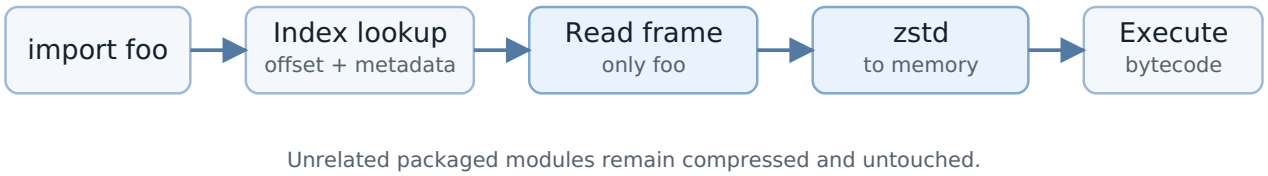


Figure 3. Pure-Python imports decompress only the requested module or resource group into memory.

Representation

The preferred representation is compiled bytecode or another directly executable Python form. Source may be retained where required. A custom import loader maps normal Python imports to indexed resources without changing lazy-import semantics.

6. Native extension model

Arbitrary Python extensions are expected to be ordinary `.so`, `.dylib`, or `.pyd` files and may have their own native dependencies. These are embedded in compressed form and materialized into an automatically managed filesystem cache only when required.

The runtime deliberately uses the platform loader rather than attempting to replace `dlopen()`, the Windows loader, or Mach-O loading semantics.

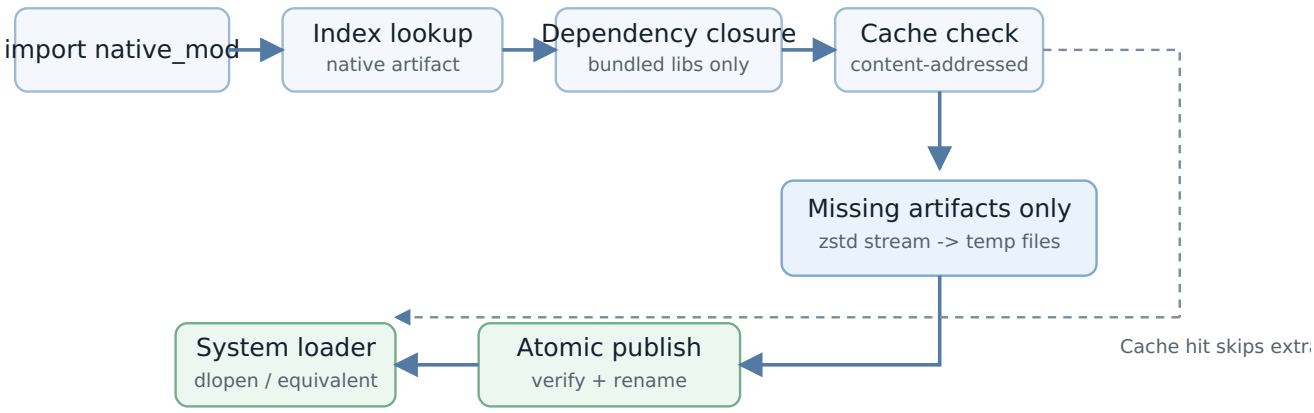


Figure 4. Native extension import: only the required dependency closure is materialized, then the normal platform loader is used.

Native dependency analysis

Build-time analysis must inspect ELF, Mach-O, or PE dependencies and classify them into system dependencies versus bundled dependencies. Bundled dependency closures are stored with the native artifact graph and extracted together when necessary.

On Linux, `$ORIGIN`, `RUNPATH`, or equivalent deterministic search paths may be used. On macOS, install names may be rewritten where needed. On Windows, DLL search directories must be controlled explicitly.

7. Content-addressed native cache

The cache is an implementation detail, automatically maintained and never manually upgraded by users. Cache identity derives from runtime ABI, platform information, and the uncompressed content identity of native artifacts.

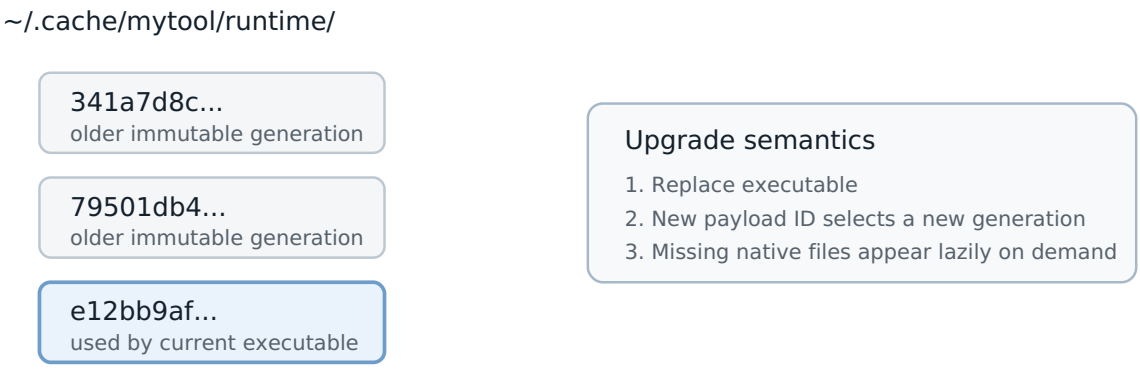


Figure 5. Cache generations are immutable. Replacing the executable transparently selects or creates the matching native cache.

Atomicity and concurrency

1. Create a temporary file within the target cache generation.
2. Stream-decompress the embedded zstd frame directly into that file.
3. Validate size and cryptographic digest.

4. Apply required permissions and synchronization where appropriate.
5. Atomically rename the completed artifact into place.

Multiple first-run processes must never observe partially decompressed native libraries. Cache corruption is self-healing: delete or replace the invalid artifact and reconstruct it from the executable, which remains the source of truth.

8. Package resources and filesystem assumptions

Ordinary package resources should behave like pure-Python modules: decompress into memory on demand. Resources that genuinely require filesystem semantics may be lazily materialized into the managed cache.

Resource	Default runtime treatment
Python module / bytecode	Decompress to memory on import.
Ordinary package data	Decompress to memory when accessed.
Native extension / shared library	Decompress to managed cache on first import, then reuse.
Filesystem-required package data	Lazily materialize to managed cache only when accessed.

9. Build pipeline

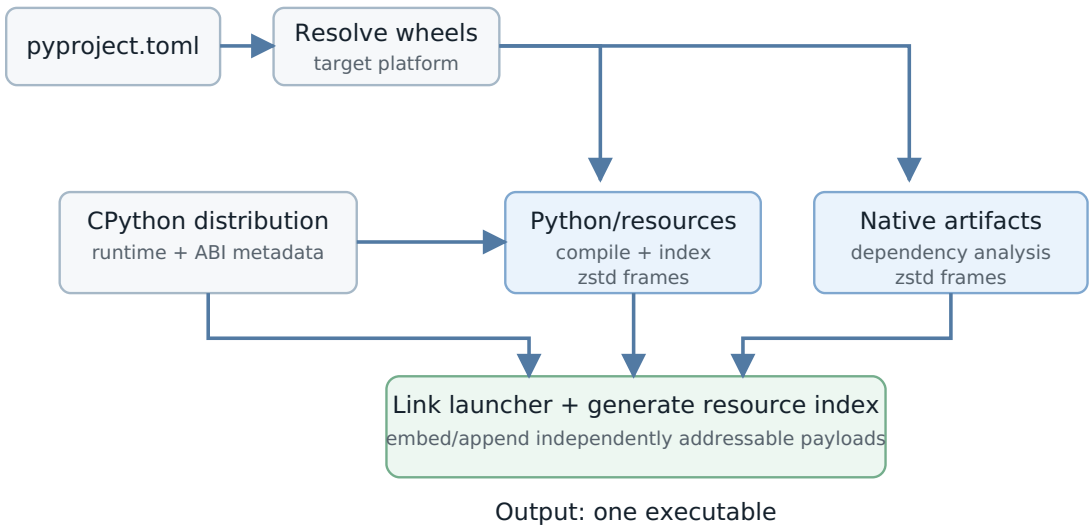


Figure 6. Build pipeline: consume existing wheels where possible, classify resources, analyze native dependencies, compress payloads, then produce one executable.

Recommended first target

Linux x86-64, glibc, CPython, and ordinary manylinux-compatible wheels. The embedded zstd decompressor should not introduce an additional external runtime dependency. Once the architecture is proven, add Linux aarch64, macOS arm64/x86-64, and Windows x86-64.

10. Compression profiles

Profile	Intent	Policy
fast	Minimum startup and import latency	Leave more small/startup-critical resources uncompressed; low zstd levels elsewhere.
balanced	Default size/performance tradeoff	Moderate zstd levels, independently addressable frames.
compact	Smaller distribution	Higher zstd levels at build time while preserving lazy decompression semantics.

Regardless of profile, the runtime never pays decompression cost for resources outside the executed import path.

11. Security and integrity

- Store a cryptographic digest for every extractable artifact.
- Validate declared uncompressed sizes before allocating or writing output.
- Prevent path traversal, symlink attacks, unsafe permissions, partial writes, and cache races.
- Do not load attacker-controlled files from writable search paths.
- Treat the executable as the immutable source of truth; cached artifacts are disposable reconstructions.

12. Non-goals

- A universally static ELF.
- One Linux binary compatible with every distribution.
- Custom ELF, PE, or Mach-O loaders as a correctness requirement.
- Zero-byte runtime cache.
- Ahead-of-time compilation of Python into machine code.
- Source-code obfuscation.
- Maximum compression ratio at the expense of startup speed.
- Guaranteed support for packages fundamentally dependent on mutable installed layouts.
- Elimination of CPython initialization overhead itself.

13. Benchmark requirements

Startup regressions are product regressions. Continuous benchmarks should compare the runtime with system Python, conventional one-file packagers, and representative Rust/Go CLIs.

Scenario	Measure
`--help` / `version`	Warm and cold startup, minimal import path.
First native import	Lazy zstd extraction, digest verification, loader cost.
Subsequent native import	Cache-hit behavior and loader cost.
Large optional package set	Verify unused dependencies do not materially affect startup CPU.
Representative real commands	Wall-clock time, CPU, peak RSS, page faults, bytes read/decompressed, imports, native libraries loaded.

14. Success criteria

A CLI containing lightweight framework code plus optional packages such as NumPy, cryptography, pydantic-core, orjson, and application modules is distributed as one executable.

Running `mytool --help` must not decompress or extract unrelated heavy packages. Executing a command that eventually imports a native package should transparently decompress only the required Python modules and native dependency closure, materialize missing native artifacts into the versioned cache, and reuse them on subsequent runs.

Replacing the executable with a newer release must transparently select or construct the matching cache generation with no user intervention.

> **Defining outcome:** from the user's perspective, deployment is still “copy executable, run executable.” Everything else - resource indexing, decompression, extraction, cache versioning, integrity checks, and cleanup - is the runtime's responsibility.

15. Design summary

Concern	Decision
Distribution	Single executable.
Python runtime	Embedded CPython.
Pure-Python modules	Indexed resources, zstd-compressed, decompressed to memory on import.
Native modules	Zstd-compressed in executable; lazily materialized to content-addressed cache.
Native loading	Normal OS dynamic loader.
Cache upgrades	Immutable generations selected by content identity; automatic.
Whole-binary compression	**Explicitly rejected.**
Unused dependencies	**No import, no decompression, no extraction.**
Primary optimization target	Fast CLI startup and low work on the selected command path.

Manifest version: initial architecture draft with zstd resource compression and lazy native cache semantics.